Assessment Brief

Module Details

Module Code	UFCFK1-15-0
Module Title	Fundamentals of Data Science
Module Tutors	Saurav Gautam
Year	2025-2026
Component/Element Number	PSA/Bi-weekly assignment/Regular
Weighting	10%

Dates

Submission Date	17-April-2026
Submission Place	Backboard
Submission Time	23:59
Submission Notes	Submit Gitlab URL

Submitted By: Adarsha Karki

Student ID: 25068584

1. Write a program to generate a NumPy array of numbers (e.g. [10, 20, 30, 40, 50]). Perform the following array operations: (5*3 = 15 marks)

   a) Find the **total sum** of elements.

   b) Calculate the **mean value** of the array.

   c) Identify the **largest and smallest values** in the array.

=> This program uses the NumPy library in Python to create a one-dimensional array of numbers. It then performs basic array operations such as calculating the total sum of elements, finding the mean (average) value, and identifying the maximum and minimum values in the array. These operations demonstrate how NumPy simplifies numerical computations in Python.

```
"""Program to generate a NumPy array and perform sum, mean, max
and min operations."""

import numpy as np

# Create array

arr = np.array([10, 20, 30, 40, 50])

print("Array:", arr)

# sum

total_sum = np.sum(arr)

print("Total Sum:", total_sum)

# mean

mean_value = np.mean(arr)

print("Mean:", mean_value)

# max and min

maximum = np.max(arr)

minimum = np.min(arr)

print("Maximum value:", maximum)

print("Minimum value:", minimum)
```

Output:

```
(.venv) erza@erza-HP-Pavi
Array: [10 20 30 40 50]
Total Sum: 150
Mean: 30.0
Maximum value: 50
Minimum value: 10
```

2. Write a program to take a list of numbers as input from the user (at least **12 elements**). Sort the list and perform slicing operations to extract elements between index ranges such as **3–6, 6–9, 4–10**. (15 marks)

=> This program takes a list of at least 12 numbers as input from the user and stores them in a list. The list is then sorted in ascending order. After sorting, slicing operations are performed to extract specific portions of the list based on given index ranges such as 3–6, 6–9, and 4–10. This program demonstrates list handling, sorting, and slicing techniques in Python.

```
"""
Program to take at least 12 numbers from user, sort them and
perform slicing operations.
"""
# Take input (at least 12 numbers)
while True:
   numbers = list(map(int, input("Enter at least 12 numbers
separated by space: ").split()))
   if len(numbers) >= 12: # check if user entered atleast 12
numbers
       break
   print("  Error: Please enter at least 12 numbers. Try again.")
numbers.sort() # sort the list
print("Sorted List:", numbers)
# slicing operations
print("Elements from index 3 to 6:", numbers[3:7]) # 3 to 6
print("Elements from index 6 to 9:", numbers[6:10]) # 6 to 9
print("Elements from index 4 to 10:", numbers[4:11]) # 4 to 10
```

Output:

```
Enter at least 12 numbers separated by space: 1 2 3 4  5 6 7 8 9 10 12 12 14
Sorted List: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 12, 14]
Elements from index 3 to 6: [4, 5, 6, 7]
Elements from index 6 to 9: [7, 8, 9, 10]
Elements from index 4 to 10: [5, 6, 7, 8, 9, 10, 12]
```

3. Create an array of random integers using NumPy, sort it, and reshape it into a **matrix of valid dimensions**. (For example, if you have an array of size 1 × 12, reshape it into 3 × 4 or 4 × 3.) [Hint: Use the reshape(row, column) method in NumPy.] (15 marks)

=> This program uses the NumPy library to generate an array of random integers. The generated array is then sorted in ascending order. After sorting, the array is reshaped into a matrix of valid dimensions such as 3×4 or 4×3 using the reshape() function. This demonstrates how NumPy can be used for generating random data, sorting arrays, and transforming them into different dimensions for further analysis.

```
"""
Program to create a random NumPy array, sort it and reshape it
into a matrix.
"""
import numpy as np

# create array
array = np.random.randint(1, 100, 12) # 12 random number from 1
to 99
print("Original array")
print(array)

# sort the array
sorted_array = np.sort(array)
print("Sorted array")
print(sorted_array)

# reshape array
reshaped_arr = sorted_array.reshape(3, 4) # reshape into 3x4
print("Reshaped array of 3x4 is")
print(reshaped_arr)
```

Output:

```
Original array
[ 3  2 14 43 63 37 79 49 33 34 37 23]
Sorted array
[ 2  3 14 23 33 34 37 37 43 49 63 79]
Reshaped array of 3x4 is
[[ 2  3 14 23]
 [33 34 37 37]
 [43 49 63 79]]
```

4. Write a program to input **two matrices** of compatible dimensions and perform operations such as **addition, subtraction, and multiplication** using NumPy. Ensure that dimension compatibility is validated before performing the operations. If the matrix sizes are mismatched, raise a proper exception. (15 marks)

=> This program takes two matrices as input from the user using the NumPy library and performs matrix operations such as addition, subtraction, and multiplication. Before performing these operations, the program checks whether the matrices are compatible in terms of their dimensions. If the dimensions are not suitable for a particular operation, the program raises an appropriate exception to prevent errors. This demonstrates the use of NumPy for matrix operations along with proper validation and exception handling.

```
"""Program to perform matrix addition, subtraction and
multiplication using NumPy with validation"""

import numpy as np

def input_matrix(name):
    # ask user for rows and cols of the matrix
    rows = int(input(f'Enter the row of {name}: '))
    cols = int(input(f'Enter the col of {name}: '))

    print(f'Enter the elements of {name}')
    elements = []

    for i in range(rows):
        row = []
        for j in range(cols):
            value = int(input(f'Enter element at row {i}, column
{j}: '))
            row.append(value)
        elements.append(row)

    return np.array(elements)

try:
    A = input_matrix("A") # passing name = A
    B = input_matrix("B") # passing name = B

    print("Matrix A:")
    print(A)
    print()
    print("Matrix B:")
    print(B)
```

```
    print()
    if A.shape == B.shape:
        print("Addition of matrix A and B is:")
        print(A + B)
        print()
        print("Subtraction of matrix A and B is:")
        print(A - B)
        print()
    else:
        print("\nError: Addition and Subtraction not possible:
Matrices must have the same dimensions.")

    if A.shape[1] == B.shape[0]:
        print("Multiplication of matrix A and B is:")
        print(np.dot(A, B))
    else:
        print("\nError: Multiplication not possible: Columns of A
must equal rows of B.")

except ValueError as e:
    print("\nError:", e)
```

Output:

```
Enter the row of A: 2
Enter the col of A: 2
Enter the elements of A
Enter element at row 0, column 0: 2
Enter element at row 0, column 1: 2
Enter element at row 1, column 0: 2
Enter element at row 1, column 1: 2
Enter the row of B: 2
Enter the col of B: 2
Enter the elements of B
Enter element at row 0, column 0: 10
Enter element at row 0, column 1: 10
Enter element at row 1, column 0: 12
Enter element at row 1, column 1: 12
Matrix A:
[[2 2]
 [2 2]]

Matrix B:
[[10 10]
 [12 12]]

Addition of matrix A and B is:
[[12 12]
 [14 14]]

Subtraction of matrix A and B is:
[[ -8  -8]
 [-10 -10]]

Multiplication of matrix A and B is:
[[44 44]
 [44 44]]
```

5. Write a program to read the CSV file **"health_data.csv"** using Pandas and plot the data using Matplotlib. Create scatter plots for the following relationships: (20 marks)
   - weight vs height
   - age vs weight
   - height vs age
   - gender vs height
   - gender vs weight

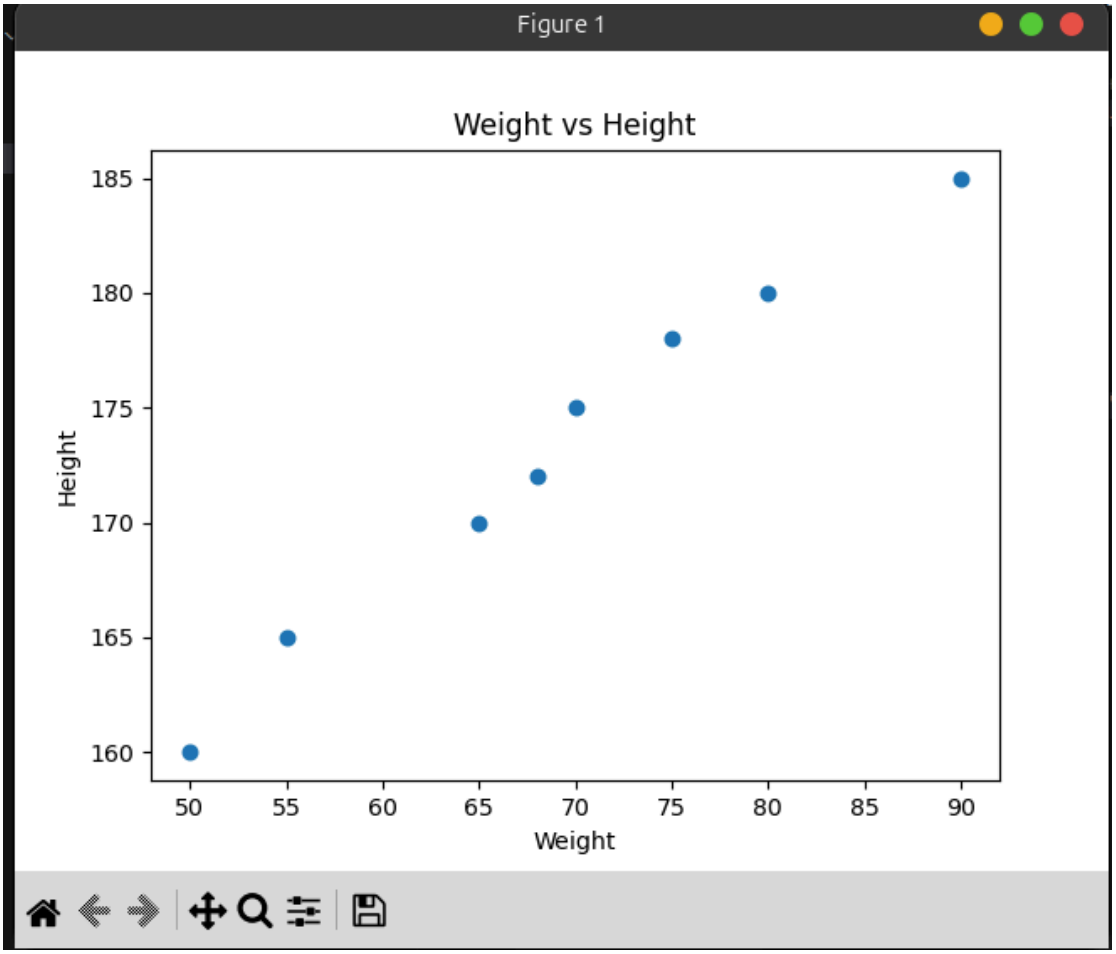
=> This program reads data from the file "health_data.csv" using the Pandas library and stores it in a DataFrame. The data is then visualized using Matplotlib by creating multiple scatter plots to show relationships between different variables such as weight vs height, age vs weight, height vs age, gender vs height, and gender vs weight. This program demonstrates data handling using Pandas and data visualization techniques using Matplotlib for exploratory data analysis.

```
"""Program to read CSV data and create scatter plots using
Matplotlib."""

import pandas as pd
import matplotlib.pyplot as plt
```

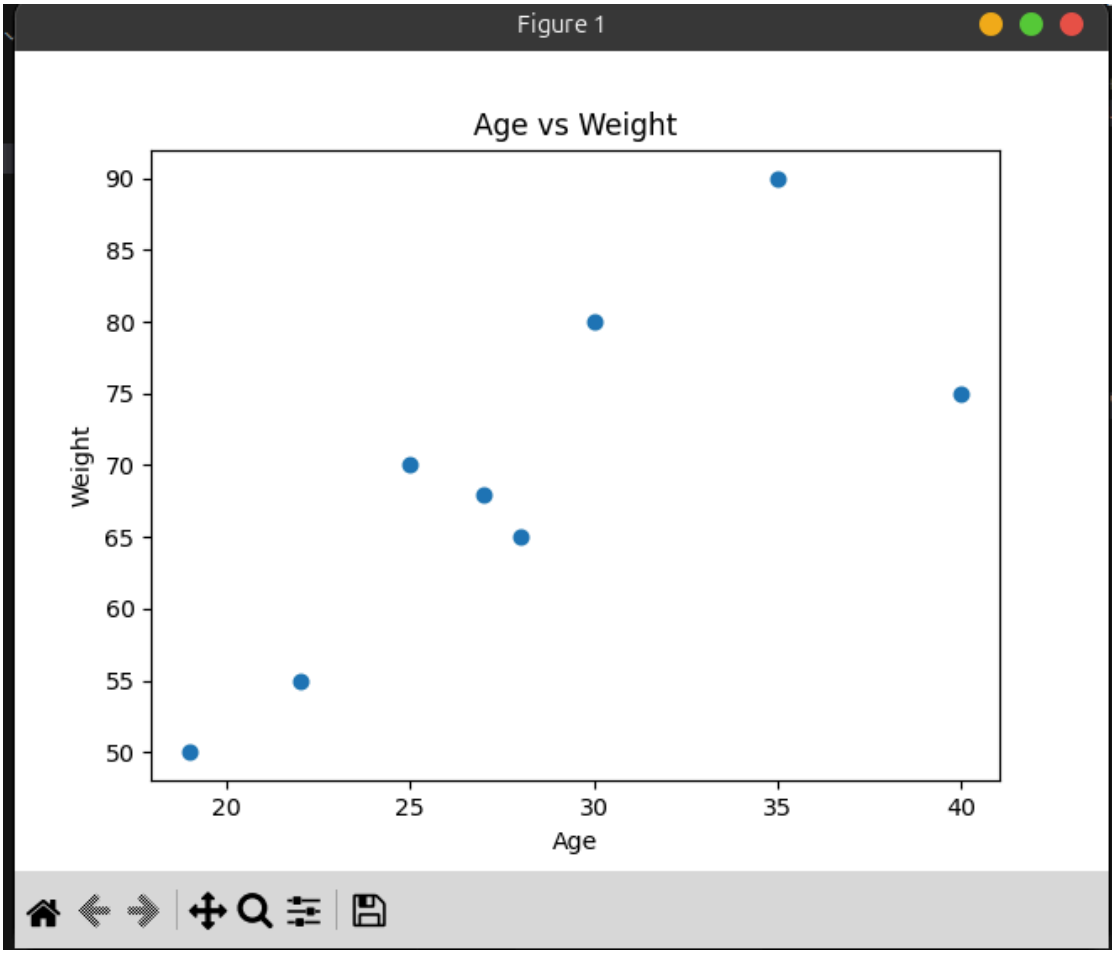
```python
file_name = 'health_data.csv'
df = pd.read_csv(file_name)

# color based on gender
colors = df['gender'].map({'Male': 'blue', 'Female': 'red'})

# weight vs height scatter diagram
plt.scatter(df['weight'] , df['height'])
plt.xlabel('Weight')
plt.ylabel('Height')
plt.title('Weight vs Height')
plt.show()

# age vs weight scatter diagram
plt.scatter(df['age'] , df['weight'])
plt.xlabel('Age')
plt.ylabel('Weight')
plt.title('Age vs Weight')
plt.show()

# height vs age scatter diagram
plt.scatter(df['height'] , df['age'])
plt.xlabel('Height')
plt.ylabel('Age')
plt.title('Height vs Age')
plt.show()

# gender vs height scatter diagram
plt.scatter(df['gender'] , df['height'] ,c=colors)
plt.xlabel('Gender')
plt.ylabel('Height')
plt.title('Gender vs Height')
plt.show()

# gender vs weight scatter diagram
plt.scatter(df['gender'], df['weight'] , c=colors)
plt.xlabel('Gender')
plt.ylabel('Weight')
plt.title('Gender vs Weight')
plt.show()
```

Output:

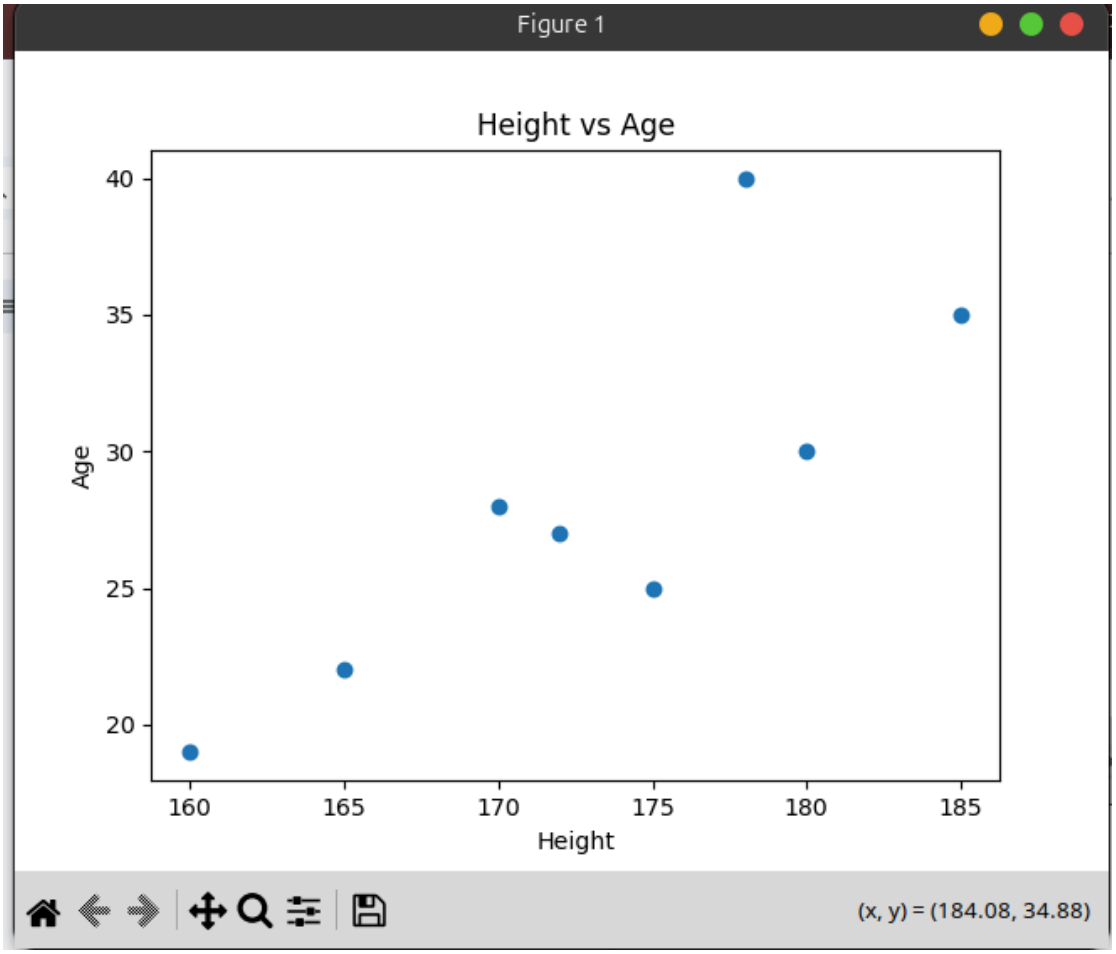
Weight vs Height



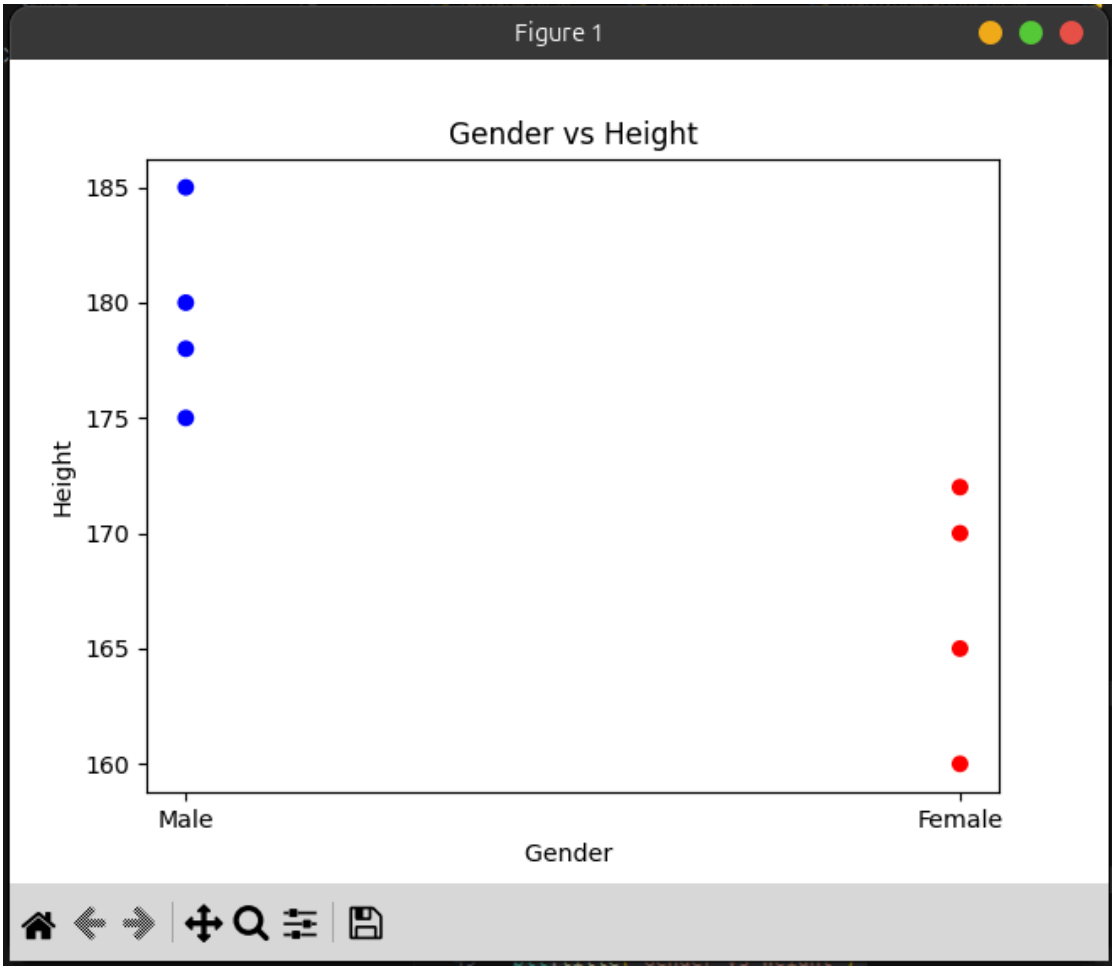
Age vs Weight



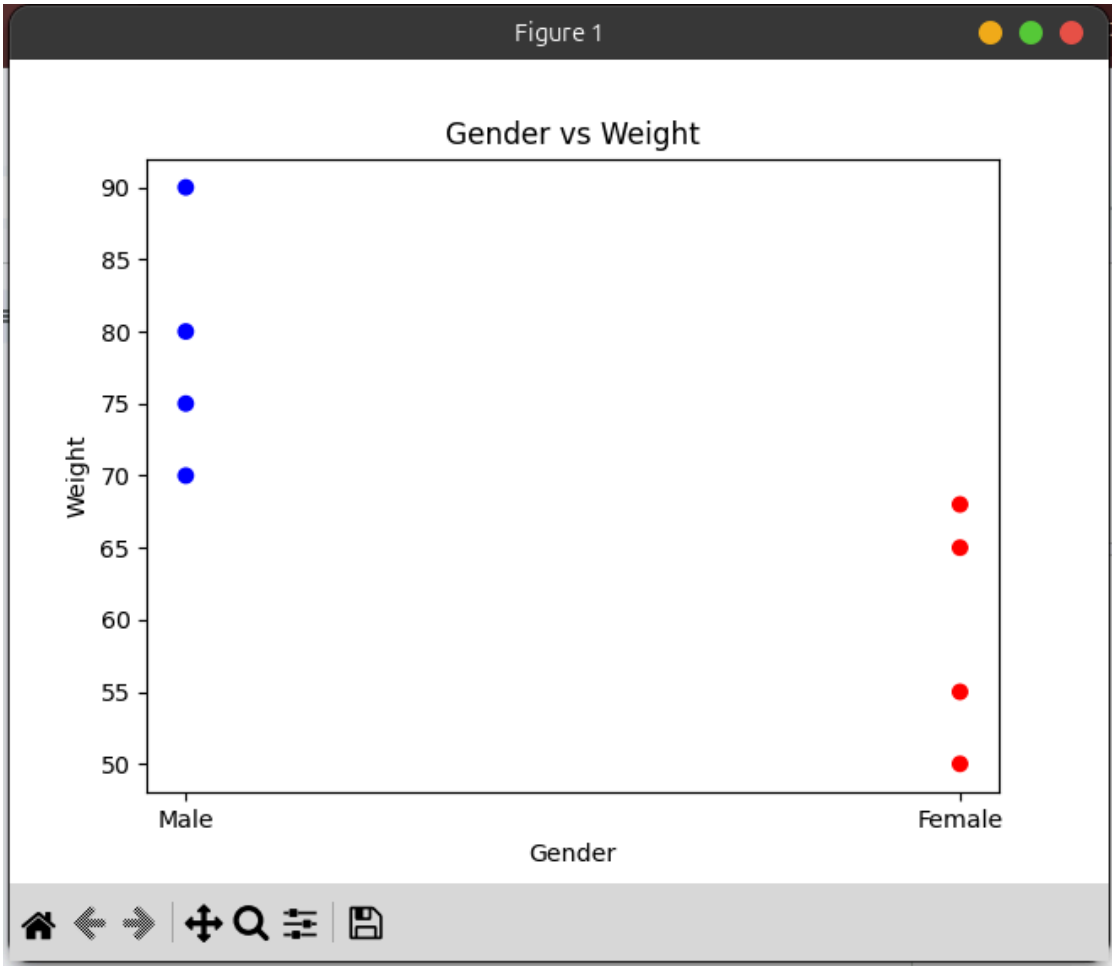
Height vs Weight



Gender vs Height



Gender vs Weight



health_data.csv

6. Read the data from the file **"health_data.csv"** into a Pandas DataFrame. Add 2 new columns: **BMI** and **Health_Status**. Populate these columns using the following conditions- (20 marks)

   - BMI = Weight / Height
   - Health_Status is based on BMI values:
     - BMI < 18.5 : Underweight
     - 18.5 ≤ BMI ≤ 24.9 : Healthy range
     - 25 ≤ BMI ≤ 29.9 : Overweight risk
     - 30 ≤ BMI ≤ 34.9 : High risk of diabetes/heart disease
     - BMI ≥ 40 : Critical health condition

=>This program reads data from "health_data.csv" using Pandas and calculates BMI using the formula BMI = weight / (height in meters)². It then classifies each record into Health_Status categories based on BMI values such as Underweight, Healthy range, Overweight risk, High risk of disease, Critical condition, and Severely Obese. Finally, it displays the updated DataFrame.

```
"""Program to calculate BMI and classify health status from CSV
data"""

import pandas as pd
df = pd.read_csv("health_data.csv")

# calculate BMI (height converted from cm to meters)
df['BMI'] = df['weight'] / ((df['height'] / 100) ** 2)

# function to get health status based on BMI
def get_health_status(bmi):
    if bmi < 18.5:
        return 'Underweight'
    elif 18.5 <= bmi <= 24.9:
        return 'Healthy range'
    elif 25 <= bmi <= 29.9:
        return 'Overweight risk'
    elif 30 <= bmi <= 34.9:
        return 'High risk of diabetes/heart disease'
    elif bmi >= 40:
        return 'Critical health condition'
    else:
        return 'Severely Obese'

df['Health_Status'] = df['BMI'].apply(get_health_status)
```

```
print(df)
```

Output:

```
(.venv) erza@erza-HP-Pavilion-Laptop-14-dv2xxx:~/python/assignme
   age  weight  height  gender        BMI    Health_Status
0   25      70     175    Male  22.857143    Healthy range
1   30      80     180    Male  24.691358    Healthy range
2   22      55     165  Female  20.202020    Healthy range
3   28      65     170  Female  22.491349    Healthy range
4   35      90     185    Male  26.296567  Overweight risk
5   40      75     178    Male  23.671254    Healthy range
6   19      50     160  Female  19.531250    Healthy range
7   27      68     172  Female  22.985398    Healthy range
(.venv) erza@erza-HP-Pavilion-Laptop-14-dv2xxx:~/python/assignme
```

health_data.csv:

```
reshape.py    slicing.py    matrixop
health_data.csv > data
You, 3 days ago | 1 author (You)
1  age,weight,height,gender
2  25,70,175,Male
3  30,80,180,Male
4  22,55,165,Female
5  28,65,170,Female
6  35,90,185,Male
7  40,75,178,Male
8  19,50,160,Female
9  27,68,172,Female
```